

Объектно-реляционная модель данных

Постреляционные базы данных
Лекция №1

Виноградова М.В.
МГТУ им. Н.Э. Баумана
2026 г.

Темы лекции

- Оргвопросы
- Поколения БД и тенденции их развития
- Особенности БД третьего поколения
- ER, реляционная модель и объектно-реляционная модели
- Поддержка сложных типов данных в языке SQL (стандарты SQL-99 и SQL-2003)
- Пользовательские типы данных (UDT) и работа с ними

Оценка

- Семестр = 70/42 (осн) + 30/0 (доп)
- Экзамен = 30/18
- Оценка
 - 60 = 3 (удв)
 - 71 = 4 (хор)
 - 85 = 5 (отл)

Баллы

контрольное мероприятие	баллы min - max	бонус (в срок)	бонус (защита)
ЛР1 - ЛР6	Вып. (3 - 5), защита (0.5 - 1)	1	
Д31	07 -- 10	1	2
Д32	07 -- 10	1	2
РК	07 -- 10	1	
все лекции	0 - 2		6
Статья	0 - 2	1	10
сумма	42 - 70	10	20

Контрольные мероприятия

КМ	Кол-во	Срок
Лабораторные Работы	6	2 нед. от нач лр
ДЗ	2	15.04 (10.05) 30.04
РК	1	14 (15) нед.
статья		30.04 (30.05)

База данных

- Структурированные данные
- Длительное (постоянное) хранение
- Под управлением СУБД

СУБД

- Язык DDL (Data Definition Language) – определяет новые базы данных(БД) и их схемы.
- Язык DML (Data Manipulation Language) – задает запросы (CRUD).
- Длительное хранение больших объемов структурированных данных.
- Многопользовательская среда.

Поколения СУБД

1. Первые БД появились в конце 60-х годов:
 - Бронирование авиабилетов; Банковская система; Корпоративные системы.
 - Файловая система + модель описания структуры
 - «иерархическая» (дерево); «сетевая» (граф)
2. Реляционные БД появляются с 70-х годов:
 - Язык SQL (Structured Query Language)
 - Отношения (таблицы)
 - Реляционная алгебра; оптимизация запросов
 - Основные до 90-х

Особенности реляционных БД

- Постоянное хранилище
- Многопользовательская работа
- Поддержка транзакций
- Восстановление
- Стандартный язык SQL
- Интеграция БД приложений

Ограничения РБД

- Объемы данных
- Требования к производительности
- Потеря согласованности
 - различие структур данных в БД и ПО
 - атомарные типы данных
 - сложность отображения составных объектов

Постреляционные БД

- Третье поколение (90-х – 2000-х)
 - Объектные,
 - Объектно-реляционные,
 - Полуструктурированные (XML).
- NoSql (с 2009)
 - Графовые,
 - Колоночные,
 - Документные,
 - Ключ-значение.
- Пространственные, мультимодельные и NewSql

Объектные и объектно-реляционные БД

- Причина появления
 - Расширение структур данных
 - Сложность сочетания программы и РБД
- Объектные БД
 - ОО язык программирования высокого уровня
 - Хранение объектов в БД
- Объектно-реляционные БД
 - Надстройка на реляционной БД
 - Сложные типы данных
 - Программирование на стороне сервера

Свойства БД третьего поколения

- Обязательные (те, которым система должна удовлетворять для того);
- Необязательные (те, которые могут быть добавлены для улучшения системы, но не являются обязательными);
- Открытые (т.е. области, где проектировщик может выбрать решение из набора одиноко приемлемых решений).

Обязательные свойства

- Поддержка сложных объектов:
 - Конструкторы из простых (атомарных);
 - Массивы, списки, множества и мультимножества, структуры;
 - Ортогональность конструкторов;
 - Операторы работы с объектами, в том числе глубокое и поверхностное копирование.
- **В ОРБД:*
 - Новые атомарные типы;
 - Объединение (union);
 - Доступ к элементам данных.

Обязательные свойства - 2

- Идентифицируемость объектов:
 - Идентичность и равенство объектов;
 - Разделяемость объектов (ссылки);
 - Изменение объекта (встроенные примитивы)
 - в программе – имя переменных (адрес).
- **В ОРБД:*
 - Встроенные UID доступны, если нет первичного ключа

Обязательные свойства - 3

- Инкапсуляция:
 - Интерфейс и реализация;
 - Модульность при разработке;
 - Скрытие свойств за операциями (ограничение доступа).
- **Проблемы:*
 - Незапланированные запросы;
 - Функции доступа.
- **Тогда в ОРБД:*
 - Правила: Триггеры и Ограничения.
 - Обновляемые представления (через функцию/ правила)

Обязательные свойства - 4

- Типы и классы:
 - Тип = интерфейс (сигнатуры операций) + реализация (скрыта от пользователя)
 - Тип для проверки при компиляции;
 - Класс = объявление (для создания и копирования) и экземпляры
- **В ОРБД:*
 - Пользовательские типы (+ операции)
 - Схема БД → набор классов/типов.

Обязательные свойства - 5

- Наследование:
 - Через подстановку (доп. Операции) – поведение;
 - Путём включения (классификация) – структура + дополнительные Свойства;
 - Через ограничение – аналогичный с уменьшением свойств и операций;
 - Через специализацию (доп. Информация).
- **В ОРБД:*
 - Наследование типов и наследование таблиц
 - Ограничения целостности

Обязательные свойства - 6

- Перезагрузка и позднее связывание :
 - Вопрос проверки типов.
- Вычислительная полнота:
 - Не ресурсная полнота.
- Расширяемость:
 - Предопределенные типы и пользовательские;
 - Создание своих типов и поддержка их системой.

Обязательные свойства - 7

- Стабильность (сохранность данных):
 - Ортогональность;
 - Неявность.
- **В ОРБД:*
 - Для различных ЯВУ (соответствие типов);
 - Кэш;
 - моделирование типов.

Обязательные свойства - 8

- Управление вторичной памятью:
 - Индексы, кластеры, буферизация,
 - выбор пути доступа, оптимизация запросов ;
 - Возможность настраивания;
 - Независимость от логической структуры.
- **В ОРБД:*
 - Методы хранения;
 - Буфер клиента/сервера.

Обязательные свойства - 9

- Параллелизм:
 - Атомарность и последовательность операций;
 - Управление совместным доступом;
 - Сериализуемость операций.
- * Восстановление после сбоя:
 - Программного;
 - Аппаратного.

Обязательные свойства - 10

- Средства обеспечения незапланированных запросов:
 - Средство высокого уровня (что, а не как);
 - Эффективно (возможность оптимизации запроса);
 - Не зависит от приложения (и пользовательских типов).
- **В ОРБД:*
 - Возможность навигации (нет оптимизации запроса)

Необязательные свойства

- Множественное наследование;
- Распределенность;
- Проверка и вывод типов:
 - Создание пользовательских из атомарным
 - Создание временных типов (автоматически)
- Проектные транзакции;
 - Протяженные или вложенные
- Поддержка версий.

Дополнительные (открытые) свойства

- Парадигма программирования:
 - Функциональное, логическое, империческое;
- **В ОРБД:*
 - Доступ из различных ЯВУ
- Однородность обработки временных и постоянных данных:
 - Тип, объект, метод и их внутреннее представление;
 - Уровень языка программирования и Уровень интерфейса.

Дополнительные (открытые) свойства - 2

- Система представлений:
 - Набор типов;
 - Конструктор типов.
- Система типов:
 - Принцип формирования типов, конструкторы.
- **В ОРБД:*
 - Создание множества:
 - Перечислением элементов (экстенциональное)
 - Запросом (интенциональное)

Особенности ОР БД

- Расширение языка SQL
- Обратная совместимость с РБД
- Клиент-серверное взаимодействие (запрос/ответ как нижний уровень коммуникации)

Вопросы создания БД

- Проектирование БД
 - Модель;
 - Структура;
 - Типы данных;
 - Связь элементов.
- Программирование приложений БД
 - На стороне клиента;
 - На стороне сервера БД.
- Распределенная обработка
 - Кластеры;
 - Фрагментация и репликация.

Модели данных

- Концептуальные:
 - ER-модель (сущность-связь);
- Логические:
 - Реляционная (SQL; нормализация);
 - Объектно-реляционная (расширение SQL);
 - Объектная (ODL);
 - Полуструктурированная (XML),
 - Агрегатные;
 - Графовая.

ER – модель

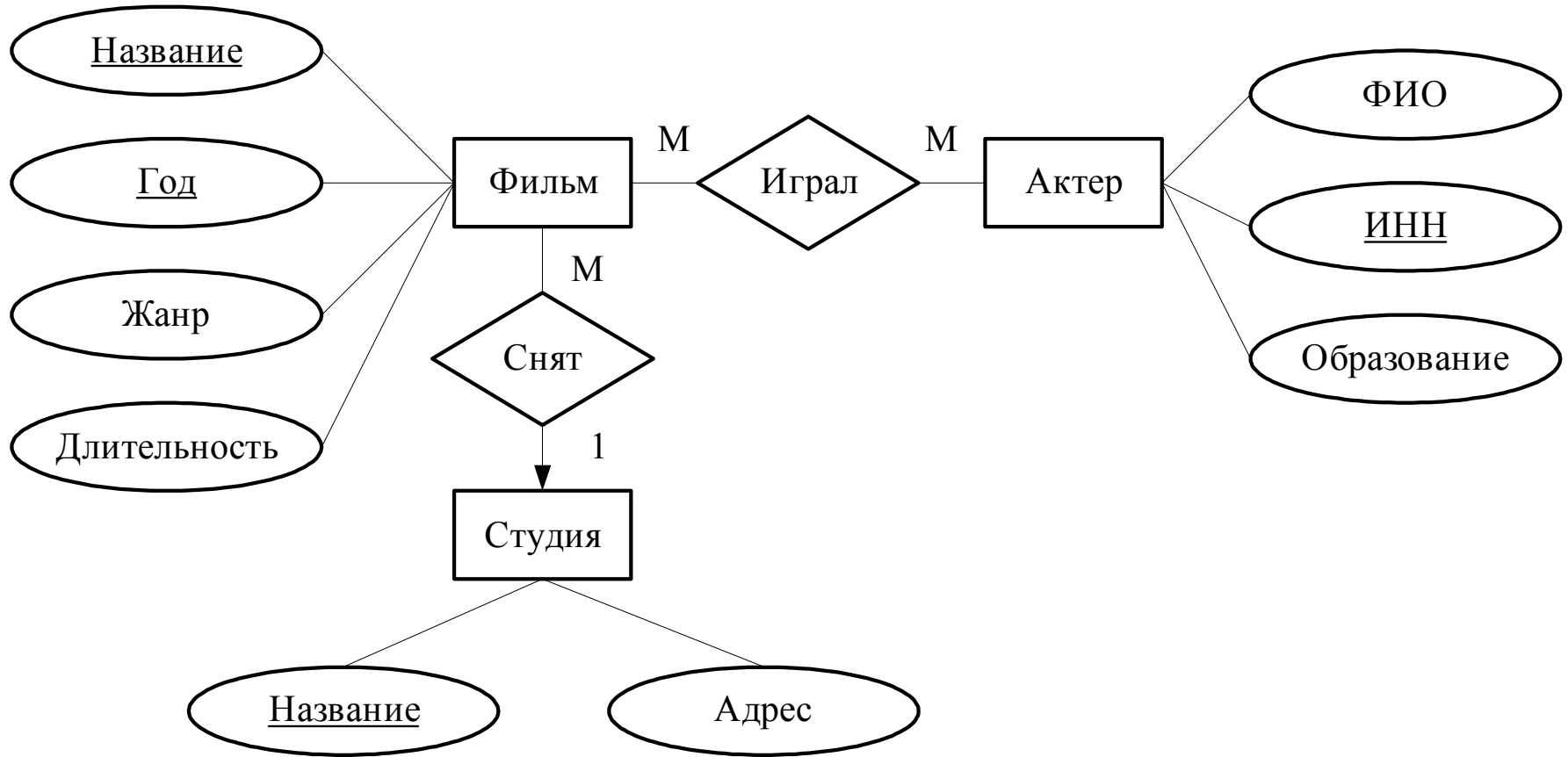
- Entity-relationship model.
- Задаёт структуру БД на концептуальном уровне.
- Объекты предметной области, их атрибуты и связи между ними.
- Использует графическую нотацию.
- Нотации Чена, Мартина, IDEF1X, Баркера и др.

Нотация Чена.

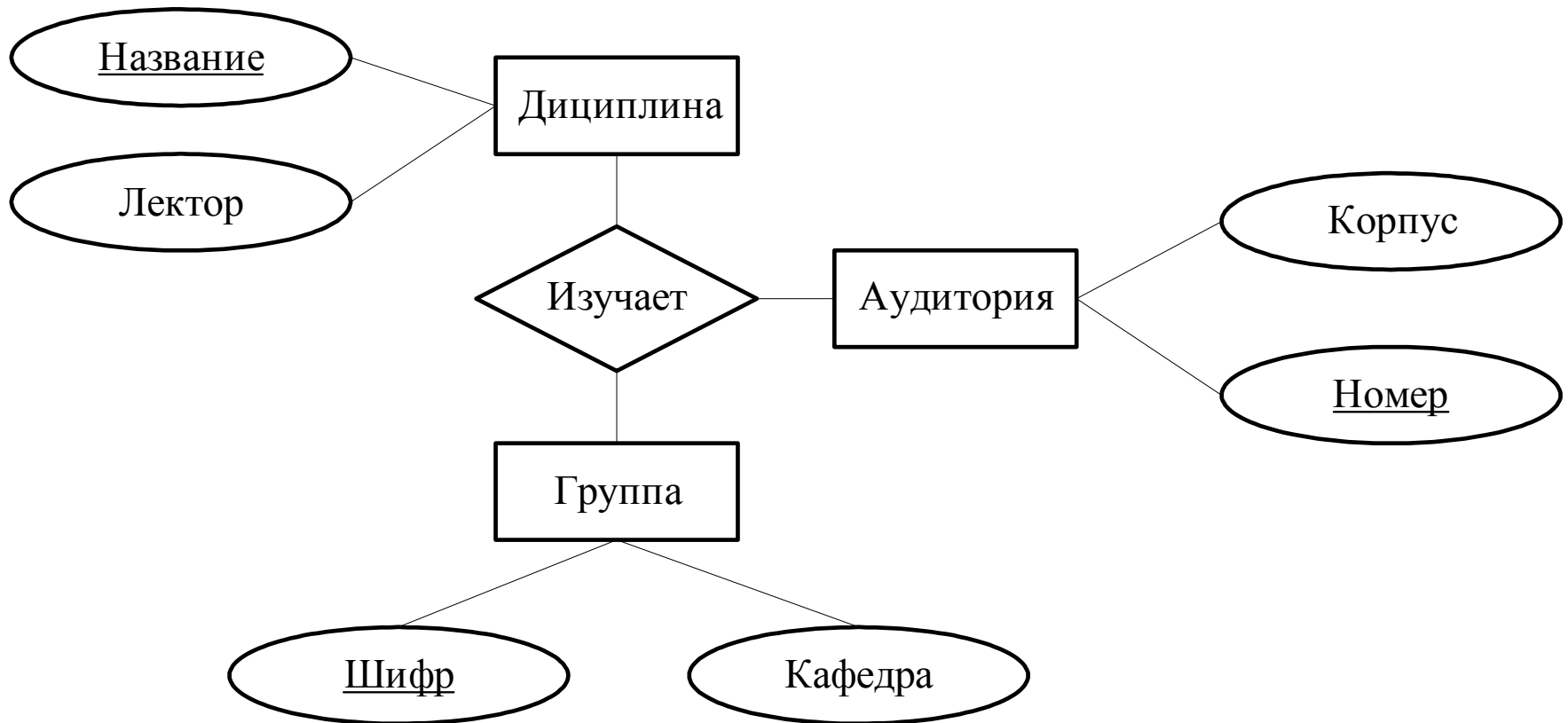
Основные элементы

Элемент модели	Графическое обозначение	Описание элемента
Сущность (набор сущностей)	 ИМЯ	Определяет объект предметной области или набор подобных объектов
Атрибут	 ИМЯ	Характеризует сущность. Может относиться к связи
Ключ (первичный ключ)	 ИМЯ	Соответствует атрибуту(ам), идентифицирующему(им) сущность
Связь	 ИМЯ	Соединяет две или более сущности. Связи могут быть 1–1, 1–М, М–1 и М–М.

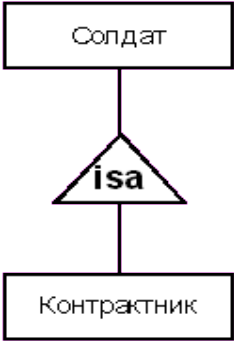
Пример простой ER-модели



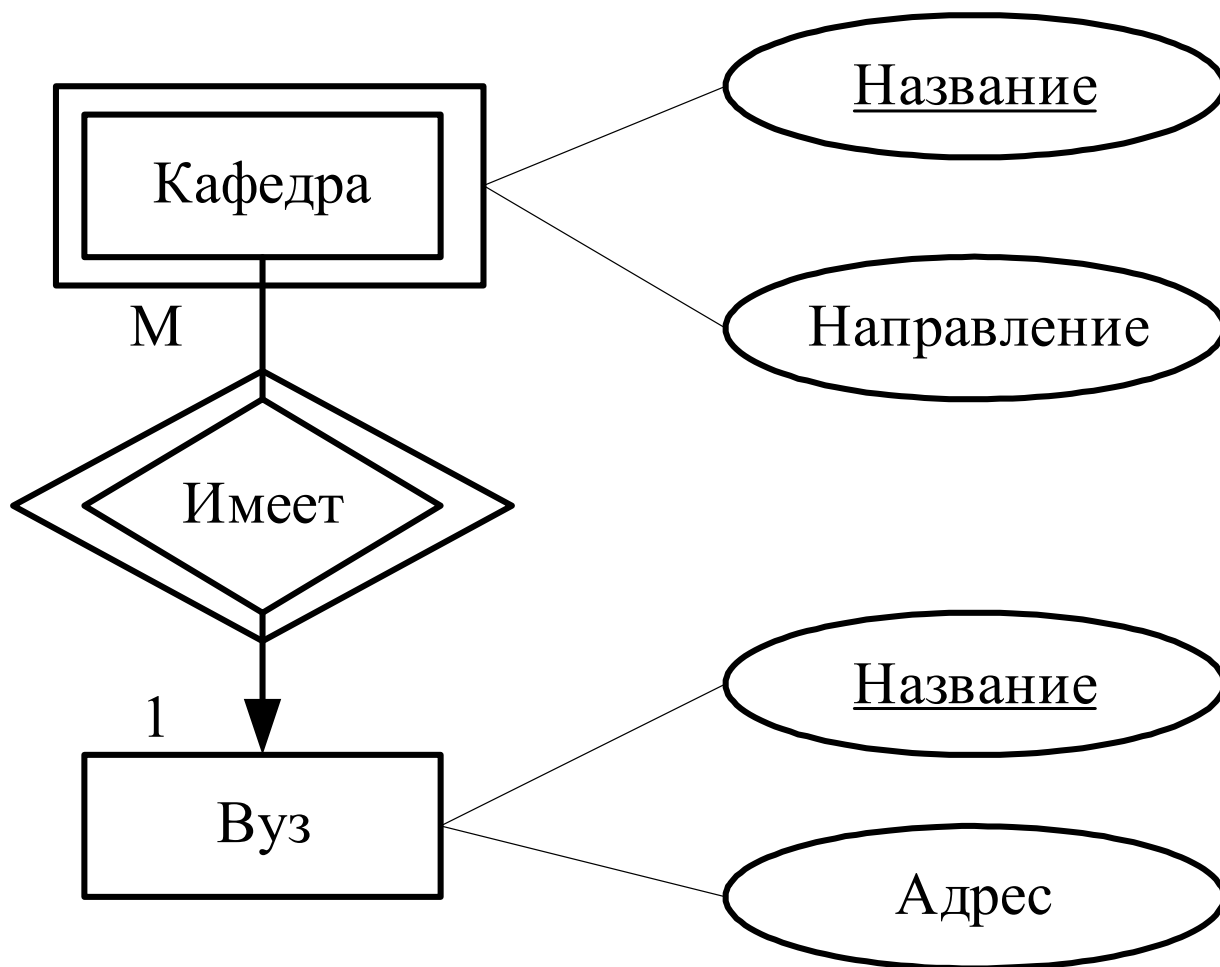
Пример тернарной связи



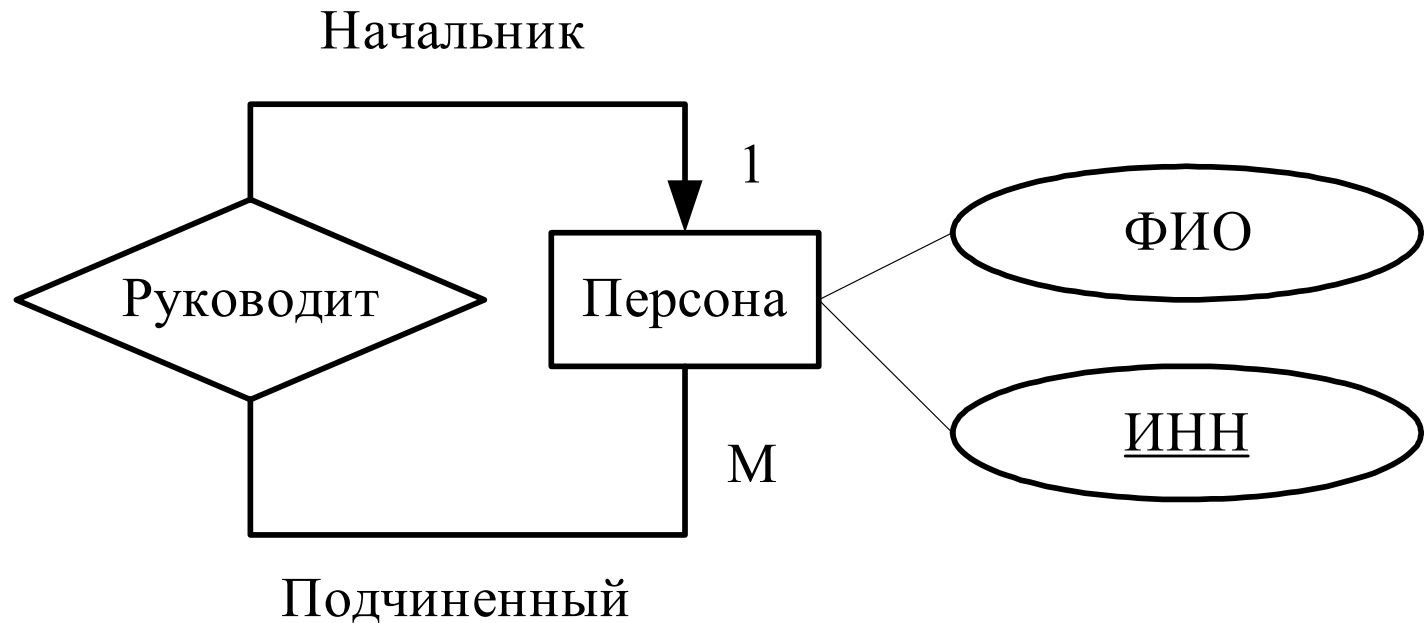
Дополнительные элементы ER-модели

Элемент модели	Графическое обозначение	Описание элемента
Слабая сущность		Не может существовать без поддерживающей сущности
Поддерживающая связь		Соединяет зависимую сущность с поддерживающими ее
Связь ISA		Расширяет базовую сущность дополнительными атрибутами. Аналог наследования

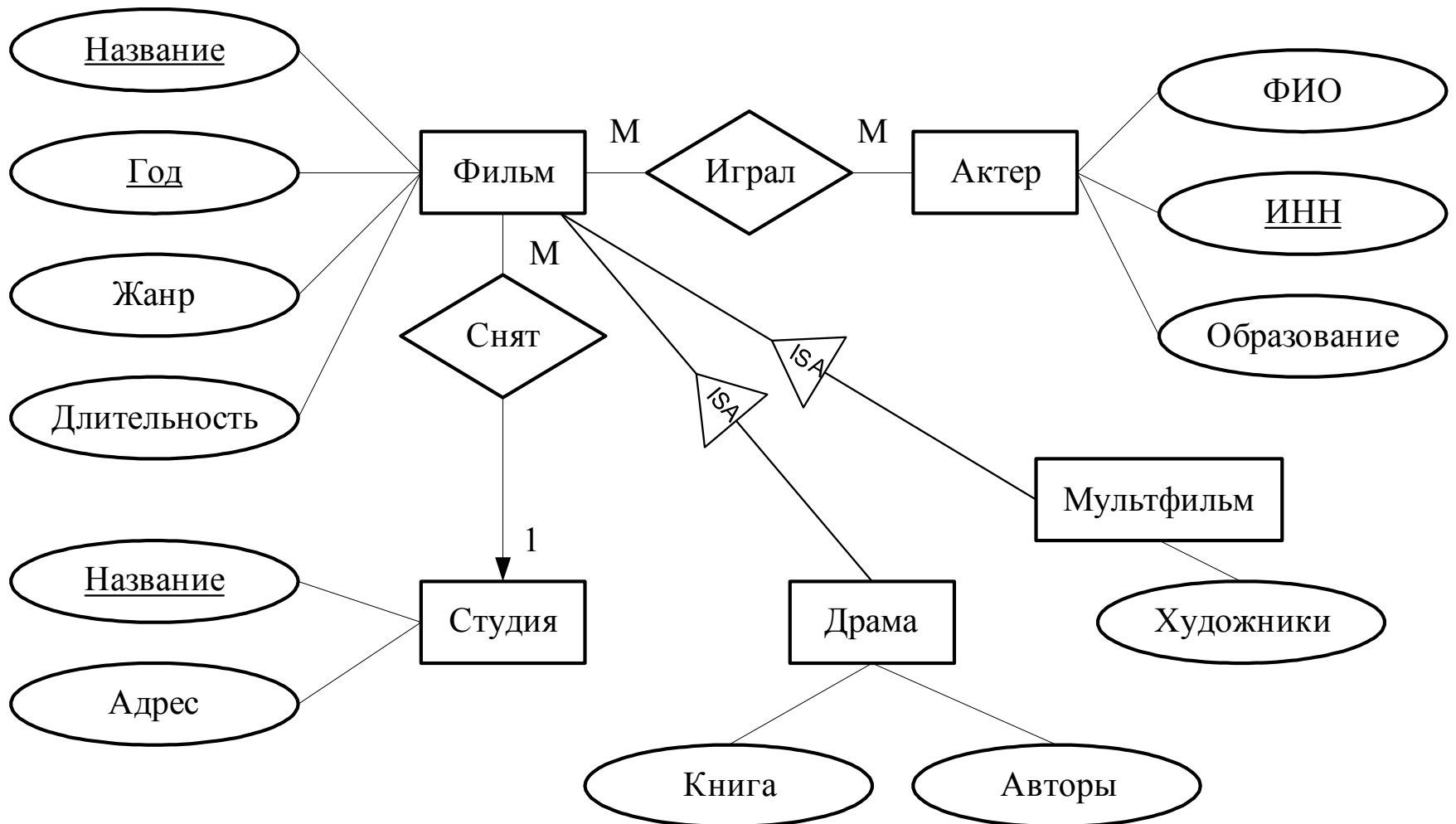
Пример поддерживающей (идентифицирующей) связи



Пример ролей связи



Пример ER-модели



Ограничения на ER-модели

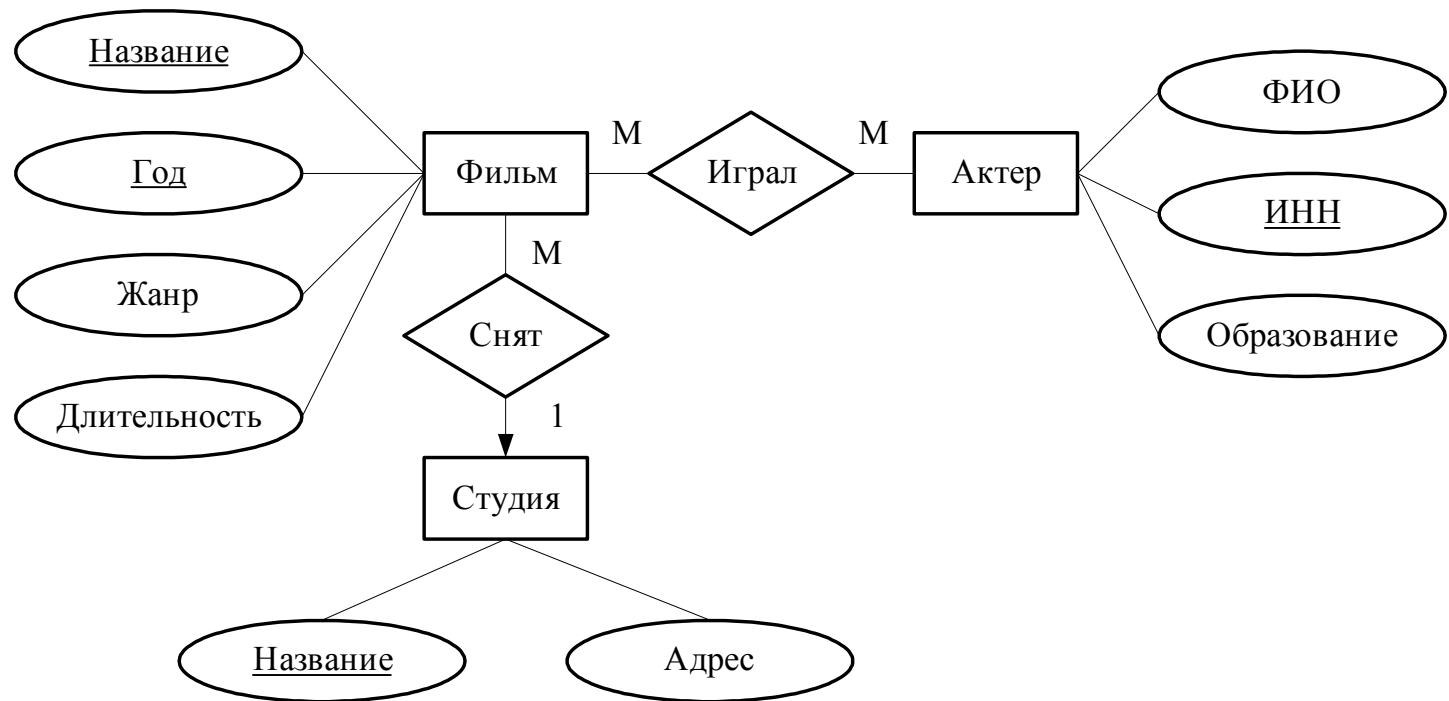
- ограничение связи типом множественности (1–M, 1–1, M–M);
- ограничения, накладываемые слабой сущностью, ключами или связью ISA;
- ограничения домена, уникальности или общего вида (например, в связи участвует не более трех сущностей)
- указывают как комментарий около элемента/связи.

Реляционная модель

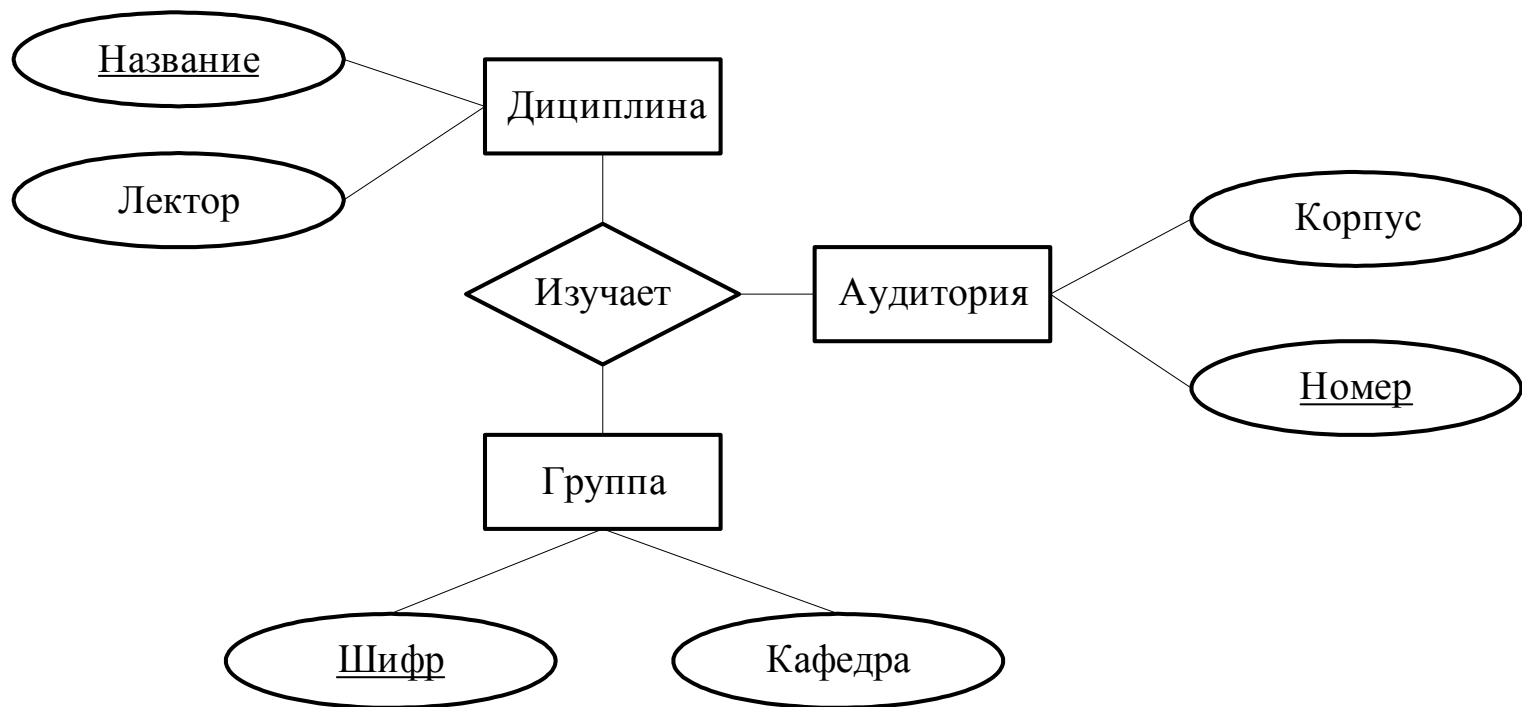
- Особенности:
 - Атомарные атрибуты;
 - Отношения;
 - Схема отношения;
 - Кортеж;
 - Домен;
 - Схема БД.
- Преимущества:
 - Нормализация / SQL / Оптимизация.

Преобразование ER модели к реляционной

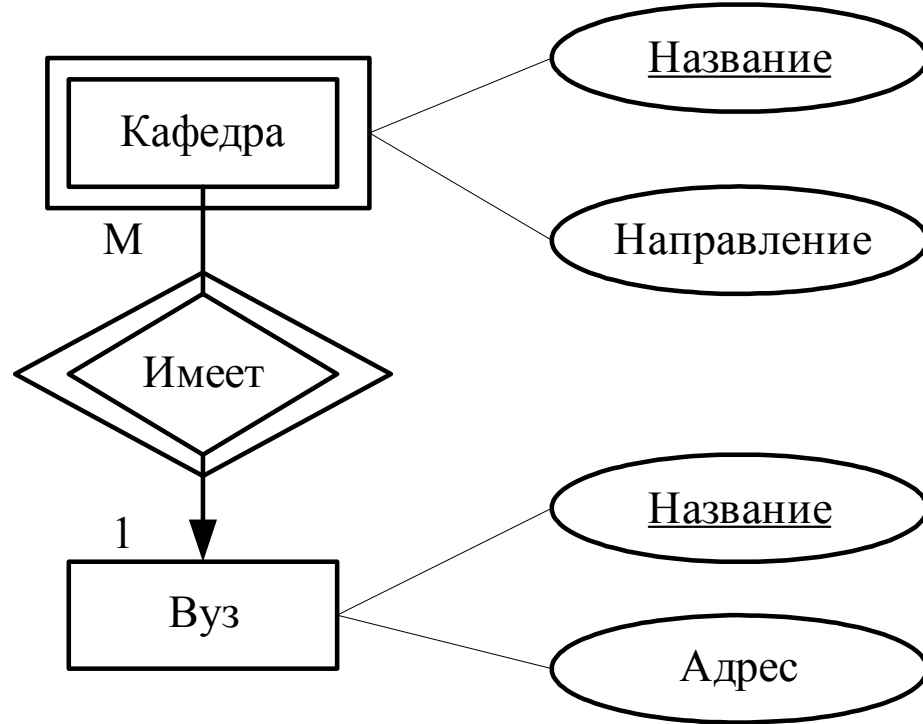
- Сущность → схема отношения (те же атрибуты).
- Атрибут → атрибут.
- Ключ → ключевой атрибут.
- Связь 1-М → внешний ключ на стороне многих.
- Связь М-М → схема отношения
 - содержит ключевые атрибуты связанных сущностей + свои атрибуты,
 - атрибуты могут быть переименованы.
- Слабая сущность → схема отношения
 - атрибуты слабой сущности + ключевые атрибуты всех поддерживающих сущностей.
- Связь слабой сущности → аналогично обычной связи



- Актёр (ИНН, Ф.И.О, Образование)
- Фильм (Название, Год, Длительность, Жанр, Студия)
- Студия (Название, Адрес)
- Актёр–Фильм (Название, Год, ИНН)



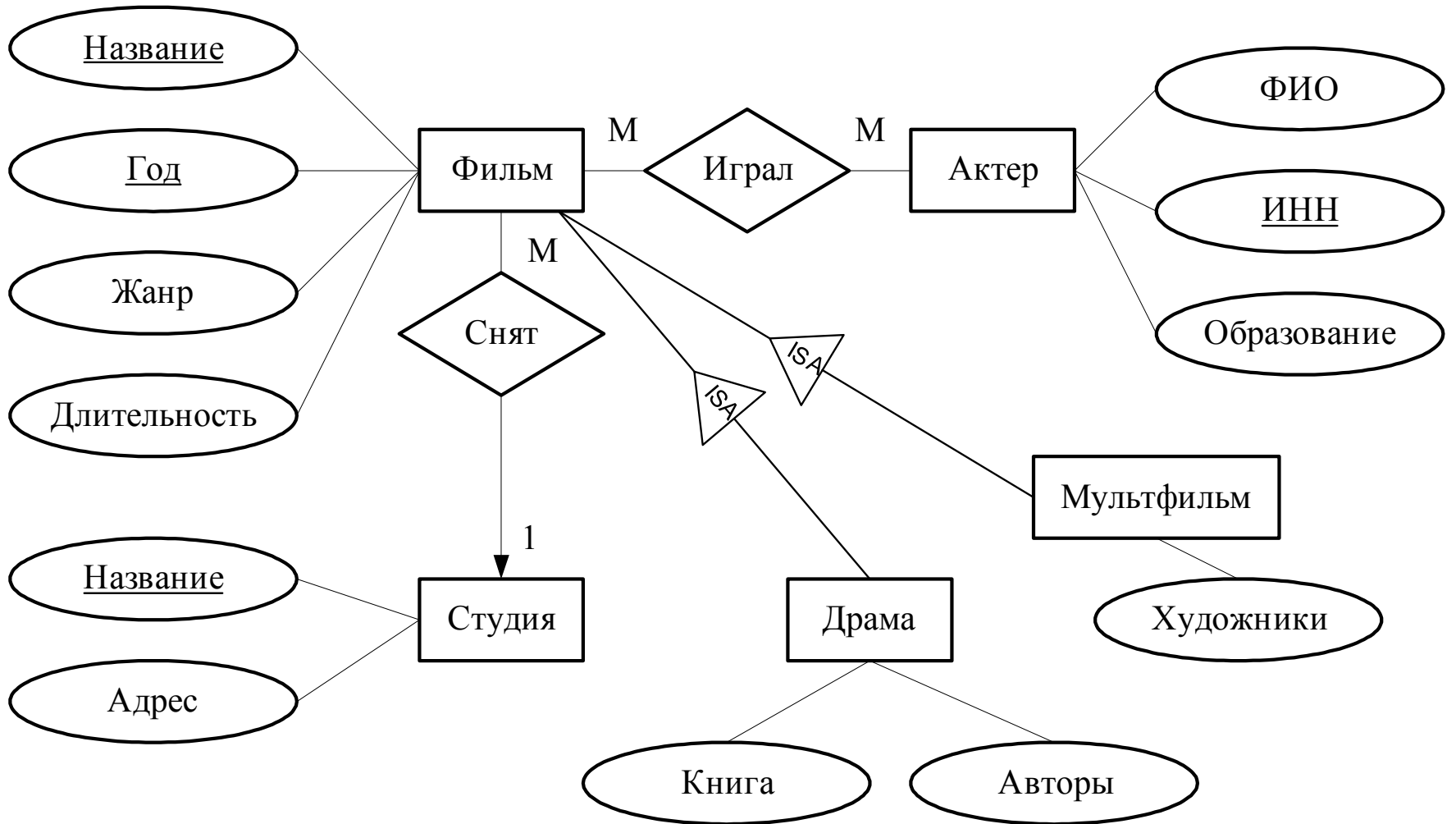
- Дисциплина (Название, Лектор)
- Группа (Шифр, Кафедра)
- Аудитория (Номер, Корпус)
- Занятие (Дисциплина, Группа, Аудитория)



- Кафедра (Название, Направление, Вуз)
- Вуз (Название, Адрес)

Преобразование иерархии сущностей (в реляции)

1. Для каждой сущности → отдельное отношение (объектный подход)
 - его атрибуты + все атрибуты предков.
2. Для каждой сущности → связанное отношение (сущностный подход)
 - его атрибуты + ключевые атрибуты корня (FK),
 - в запросах используют соединение.
3. Одно отношение для всех сущностей иерархии (подход пустых значений)
 - со всеми возможными атрибутами,
 - для отсутствующих значений – NULL.



- При *сущностном подходе*
 - Фильм (Название, Год, Длительность, Жанр, Студия)
 - Мультфильм (Название, Год, Художники)
 - Драма (Название, Год, Книга, Авторы)
- При *объектном подходе*
 - Фильм (Название, Год, Длительность, Жанр, Студия)
 - Мультфильм (Название, Год, Длительность, Жанр, Студия, Художники)
 - Драма (Название, Год, Длительность, Жанр, Студия, Книга, Авторы)
- При *подходе пустых значений*
 - Фильм (Название, Год, Длительность, Жанр, Студия, Художники, Книга, Авторы).

Описание реляционной модели на языке SQL

```
CREATE TABLE имя_таблицы  
(  
    Имя_поля тип    [размер]    [ограничение_поля]  
                                [ DEFAULT значение ] ,  
    ....  
    Ограничения_целостности ,  
    ....  
);
```

Атомарные типы

- **integer** — целое поле;
- **char** (количество_символов) — строка из фиксированного количества символов;
- **varchar** (число_символов) — строка символов, размером не более указанного;
- **smallint** — целое число;
- **float** — действительное число;
- **date** — дата;
- **time** — время.

Ограничения

- Для столбца:
 - **NOT NULL** — значение всегда не пустое ;
 - **UNIQUE** — все значения столбца должны быть уникальны;
 - **PRIMARY KEY** — первичный ключ и уникальность;
 - **FOREIGN KEY** — внешний ключ,
 - **CHECK** (условие) — условие для проверки значения столбца. В скобках м.б. условие, выборка, запрос.
- Ограничения для таблицы:
 - **CHECK** — ограничение на значения полей кортежа;
 - **PRIMARY KEY** — первичный ключ,
 - **FOREIGN KEY** — ограничение ссылочной целостности.

Ограничения внешнего ключа

FOREIGN KEY (имя_столбца_внешнего_ключа [,...n])

REFERENCES имя_родительской_таблицы

(имя_столбца_родительской_таблицы [,...n]),

[ON UPDATE

*{CASCADE| SET NULL |
SET DEFAULT | NO ACTION}*

[ON DELETE

*{CASCADE | SET NULL |
SET DEFAULT | NO ACTION}*

Пример описания таблиц на SQL

```
CREATE TABLE Stud
(
    sid INTEGER PRIMARY KEY UNIQUE,
    sname VARCHAR(50) NOT NULL,
    addr VARCHAR(300)
);

CREATE TABLE Actor
(
    inn CHAR(10) PRIMARY KEY,
    fio VARCHAR(200),
    edu VARCHAR(50)
);
```

Пример описания таблиц на SQL

```
CREATE TABLE Film
(
  title varchar(100),
  year integer,
  length integer NOT NULL,
  type char(20) CHECK( type IN ('комедия', 'драма',
                                'фантастика') ),
  stud integer REFERENCES Stud(sid)
      ON UPDATE CASCADE
      ON DELETE SET NULL,
  PRIMARY KEY (title, year),
  CHECK ( length>0 AND year>1894 )
);
```

Пример описания таблиц на SQL

```
CREATE TABLE FA
(
    act CHAR(10) NOT NULL REFERENCES Actor(inn)
        ON DELETE CASCADE
        ON UPDATE SET NULL,
    fname VARCHAR(50) NOT NULL,
    fyear INTEGER NOT NULL,
    FOREIGN KEY(fname, fyear) REFERENCES
        Film(name, YEAR)
        ON DELETE CASCADE
        ON UPDATE SET NULL,
    PRIMARY KEY(act, fname, fyear)
);
```

Объектно-реляционная модель позволяет

- хранить и обрабатывать сложные типы данных (структуры и коллекции);
- использовать пользовательские типы данных (UDT);
- включать в пользовательские типы данных атрибуты и методы;
- поддерживать уникальные идентификаторы кортежей и ссылки на них;
- определять иерархию (наследование) типов данных.

Типы данных ORM

- атомарные;
- структуры, содержащие набор именованных полей;
- массивы, хранящие упорядоченный набор элементов;
- множества и мультимножества, хранящие неупорядоченный набор элементов;
- пользовательские типы данных;
- ссылки на объекты (кортежи) пользовательских типов данных.

Элементами структур и коллекций могут быть любые типы данных, в том числе составные.

Работа с массивом в SQL-1999

Create table Tab

```
(  
    X тип ARRAY[количество элементов];  
);
```

- Значение по умолчанию: NULL или пустой - ARRAY[]
- Обращение по индексу
SELECT x[1] FROM tab;
- Изменение элемента/массива
INSERT INTO tab(x) VALUES (ARRAY[10,20,30]);
- Преобразование массива в таблицу
**SELECT r.id, a.name
FROM tab1 AS r, unnest(tab.x) as a(name)**
- Определить длину массива
- Добавить/удалить/проверить наличие элемента
- Конкатенация двух массивов

Структуры в SQL-1999, SQL-2003

Имя_поля ROW (назв тип[опции],)

```
CREATE TABLE tab(  
    a int,  
    b row(F1 int, F2 varchar(12) )  
);
```

- Обращение к полю
- Обращение к структуре

```
INSERT INTO tab(a,b) VALUES (10, (20, 'aa'));
```

```
INSERT INTO tab(a,b) VALUES (10, ROW(20, 'aa'));
```

```
SELECT a, b, b.F2 from tab WHERE b.F1>0 ;
```

Работа с коллекциями в SQL-2003

```
CREATE TABLE tab(  
    C1 INT,  
    C2 INT multiset,  
    C3 row(F1 INT, F2 INT) multiset  
);
```

Конструкторы:

- Пустой – multiset()
- Перечисление – multiset(1,3,7,8)
- Из запроса –
multiset(SELECT col FROM tab1 WHERE...)

Работа с коллекциями в SQL-2003

- Операции

- **cardinality**(коллекция) → количество элементов
- **set**(коллекция) → преобразование в множество
- **element**(коллекция) → преобразовать коллекцию из 1 элемента в элемент
- **unnest**(коллекция) as tab(c1, c2, ...) → преобразовать коллекцию в виртуальную таблицу tab(c1, c2, ...)
- ($\cup$, $\cap$ all по умолчанию)

union all
коллекция1 **multiset intersect** коллекция2
except distinct

- Агрегация

- **collect**(значения) → преобразовать в коллекцию
- **fusion**(значения) → $\cup$ в группе
- **intersection**(значения) → $\cap$ в группе

Сравнение коллекций

- Значение **[not] member [of]** коллекция
→ проверяет вхождение элемента в коллекцию
- коллекция1 **[not] submultiset [of]** коллекция2
→ Проверяет $\text{коллекция1} \subseteq \text{коллекция2}$
- коллекция **IS [not] A SET**
→ проверяет на множество

Пользовательские типы данных

- Содержат атрибуты любых типов, кроме самих себя.
- Методы пользовательских типов данных:
 - **конструктор** — при создании объекта данного типа;
 - **метод экземпляра** — вызывают для конкретного объекта и имеет доступ к его полям;
 - **метод класса (статический)** — применяют не к экземпляру, а ко всему классу.
- Иерархия типов на базе наследования
 - Производный тип наследует все атрибуты и методы базового типа и может добавлять собственные.
 - возможно переопределение методов, поддерживаются виртуальные вызовы и полиморфизм.

Структурный тип (SQL-1999)

CREATE TYPE Название [**UNDER** базовый тип]
AS

тип |

(атрибут тип [**SCOPE** ...] [**DEFAULT** X], ...)

[[**[NOT] INSTANTIABLE**]

[**[NOT] FINAL**

[

REF IS SYSTEM GENERATED -- по умолчанию

| **REF USING** тип -- создается приложением

| **REF USING** (атр 1, атр. N) -- на основе

атрибутов

]

[cast ...]

СПИСОК МЕТОДОВ;

Объявление метода типа

[OVERRIDING]

[INSTANCE | STATIC | CONSTRUCTOR]

METHOD **название** (атрибут тип, атрибут тип)
RETURNS тип

[self AS result]

[self AS locator]

[parameter style sql | java | ...]

[[NOT] deterministic]

[NO SQL | CONTAINS SQL | reads sql data | modifies sql data]

[return NULL on NULL input | called on NULL input]

User Defined Type (UDT)

```
CREATE TYPE Addr AS
```

```
(
```

```
    city CHAR(20),
```

```
    street VARCHAR(100)
```

```
)
```

```
method fullad() RETURNS VARCHAR(130);
```


Методы типов

- Тело метода описывают отдельно:

```
CREATE method fullad() RETURNS  
                                VARCHAR(130)
```

```
FOR Addr
```

```
BEGIN
```

```
    RETURN SELF.city || SELF.street
```

```
END;
```

Для ограничения доступа к атрибутам пользовательских типов данных применяются следующие методы автоматической генерации объектно-реляционной СУБД:

- **метод-генератор**, т. е. конструктор;
- **метод-обозреватель** — вызывается при чтении значения атрибута;
- **метод-модификатор** — вызывается при записи значения атрибута.

Типизированные таблицы

- на основе пользовательского типа,
- каждая строка содержит один объект пользовательского типа.
- Ключи и ограничения целостности относятся к таблице, а не к пользовательскому типу.
- Для последующей ссылки на кортеж типизированной таблицы при ее создании определяют специальный идентификатор (ИД), который генерируется системой или основан на первичном ключе.

Определение типизированных таблиц

- Типизированные таблицы :

CREATE TABLE название **OF** UDT_тип
[**UNDER**
базовая_таблица]
(
[ограничения | тип_ссылки]
);

- Тип ссылки :

REF IS название_столбца
{ **SYSTEM GENERATED**
| **USER GENERATED** |
DERIVED }

Пример типизированной таблицы

```
CREATE TYPE ActorT AS
```

```
(  
    inn char(10),  
    fio ROW ( f varchar(10), i varchar(10), o varchar(10)),  
    edu varchar(15)  
);
```

```
CREATE TABLE Actor OF ActorT
```

```
(  
    PRIMARY KEY (inn),  
    REF IS Ida system generated  
);
```

Ссылки на кортежи

- Вместо внешних ключей используют ссылки.
- Внутренняя реализация ссылки не раскрывается.
- $A(*R1)$ — атрибут A , являющийся ссылкой на кортеж отношения $R1$;
- $B(\{ *R1 \})$ — атрибут B , являющийся множеством ссылок на кортежи отношения $R1$.
- Организация (Название, Адрес)
- Персона (Фамилия, Работа($\{ *Организация \}$))

Пример ссылок

```
CREATE TYPE StudT AS  
(  
    sname VARCHAR(50),  
    addr Addr  
);
```

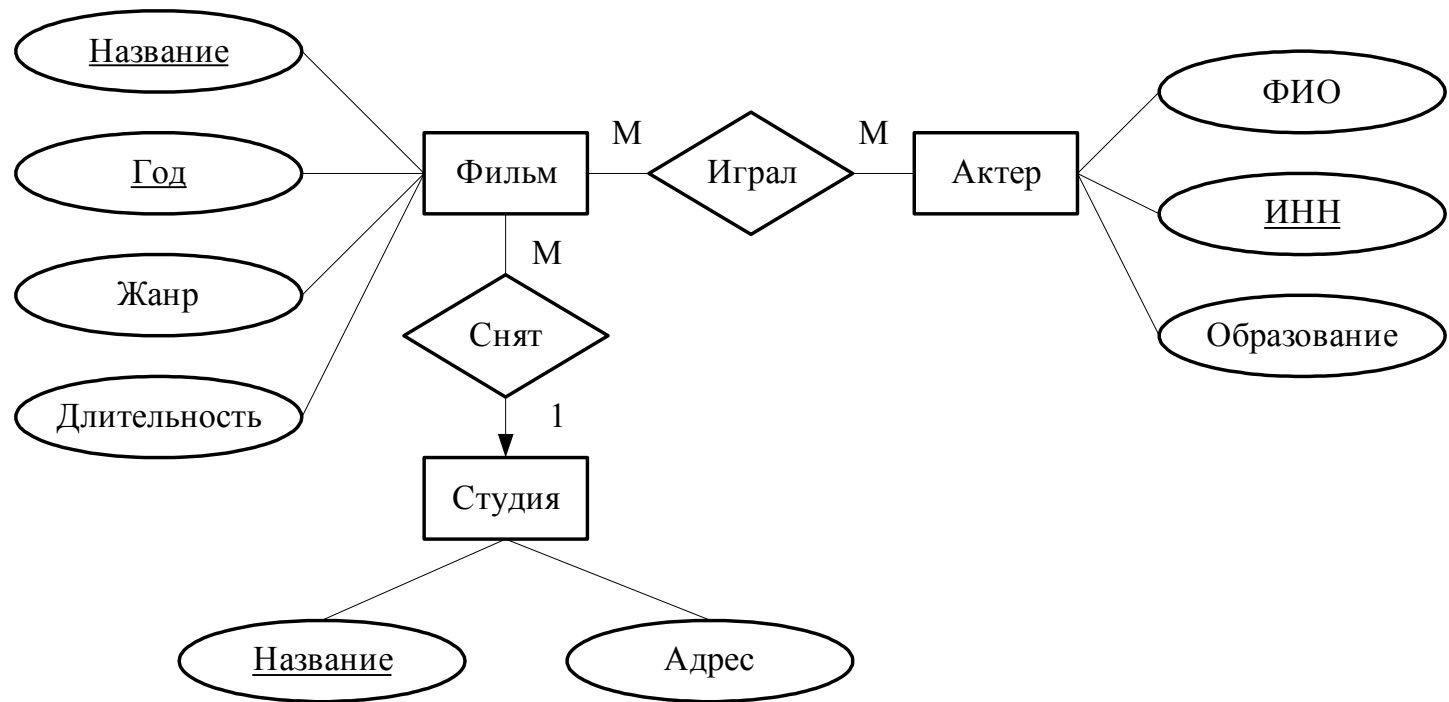
```
CREATE TYPE FilmT AS  
(  
    title varchar(100),  
    year integer,  
    length integer,  
    type char(20) ,  
    stud REF(StudT) SCOPE Stud  
);
```

Пример ссылок - 2

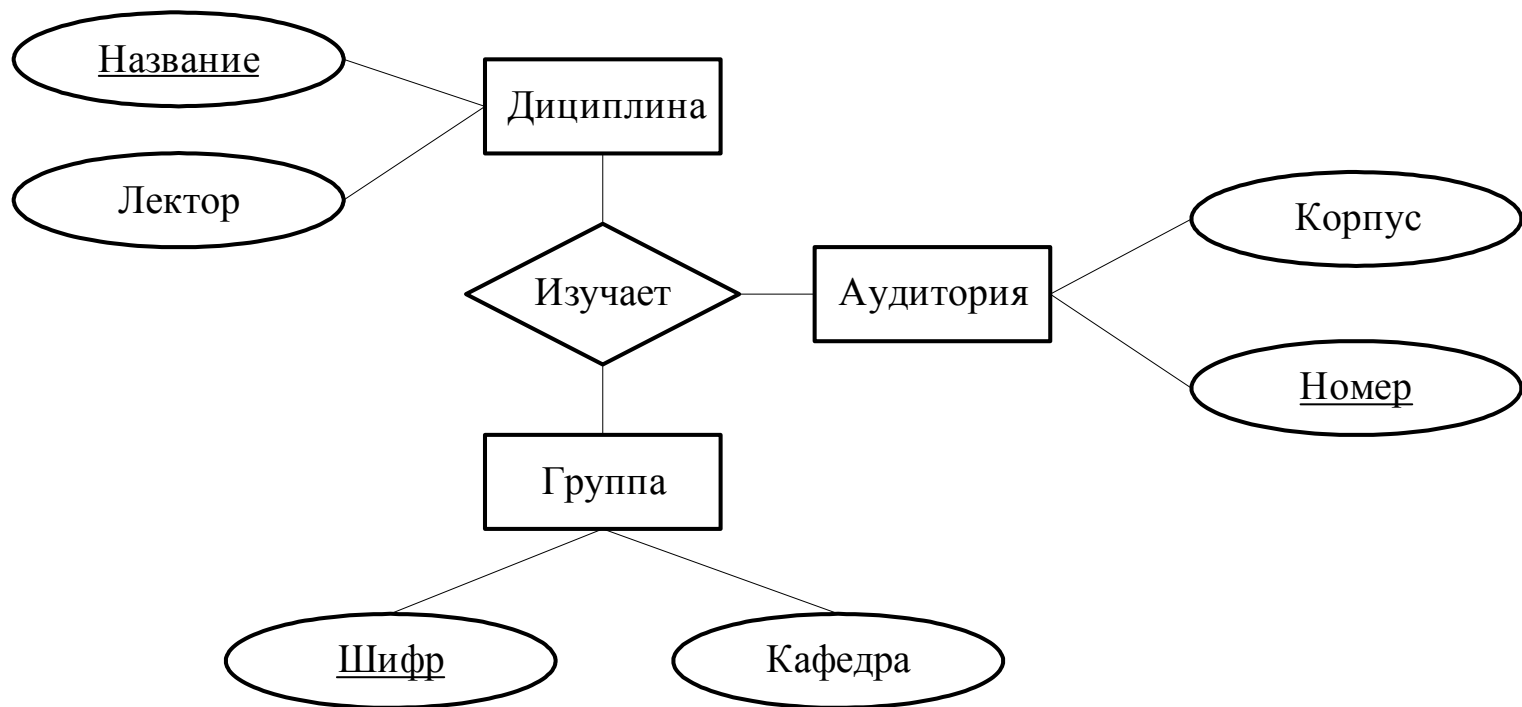
```
CREATE TABLE FA  
(  
    act REF(ActorT) SCOPE Actor,  
    film REF(FilmT) SCOPE TF  
);
```


Преобразование ER-модели в объектно-реляционную

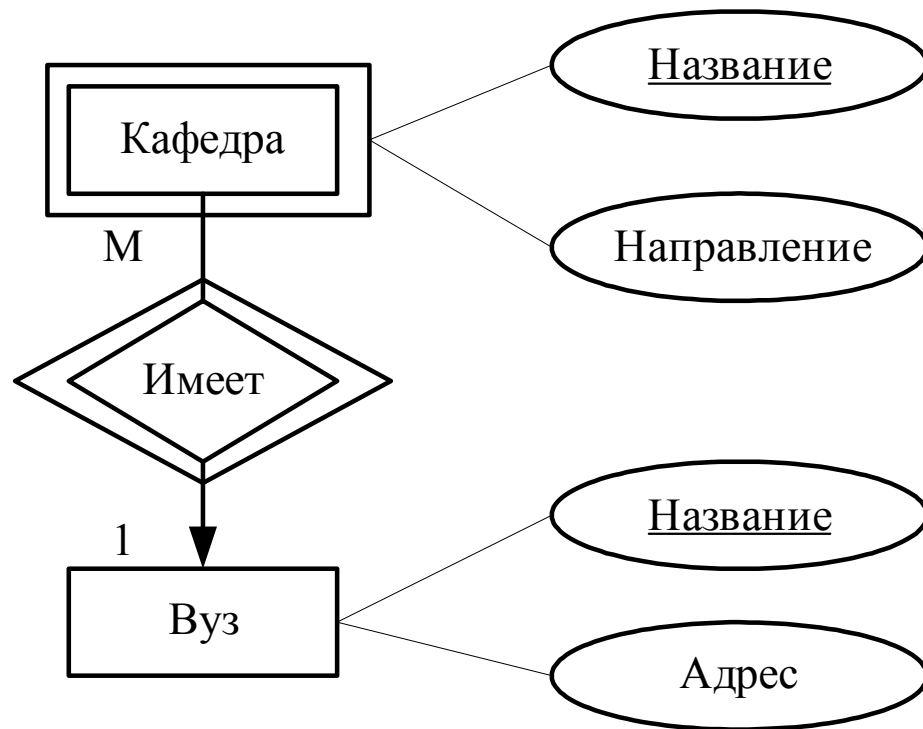
- сущность — в отношение;
- атрибут сущности — в атрибут отношения, м.б.сложные типы;
- ключ сущности — в ключ отношения;
- связь 1—М — в ссылку со стороны многих (М) на кортеж таблицы со стороны одного (1);
- связь М—М — в отношение, содержащее ссылки на кортежи связанных таблиц;
- N -арная связь ($N > 2$) — в отношение, содержащее ссылки на кортежи связанных таблиц;
- слабая сущность и поддерживающие ее связи — в отношение, содержащее атрибуты слабой сущности и ссылки на кортежи всех поддерживающих ее сущностей. Ее ключ - совокупность собственного ключа и ссылок на поддерживающие ее сущности.



- Актёры (ИНН, Ф.И.О (Фамилия, Имя, Отчество), Образование)
- Фильмы (Название, год, Длительность, Жанр, Студия (*Студии))
- Студии (Название, Адрес (Город, Страна))
- Актёр–Фильм (Фильм (*Фильмы), Актер(*Актеры))



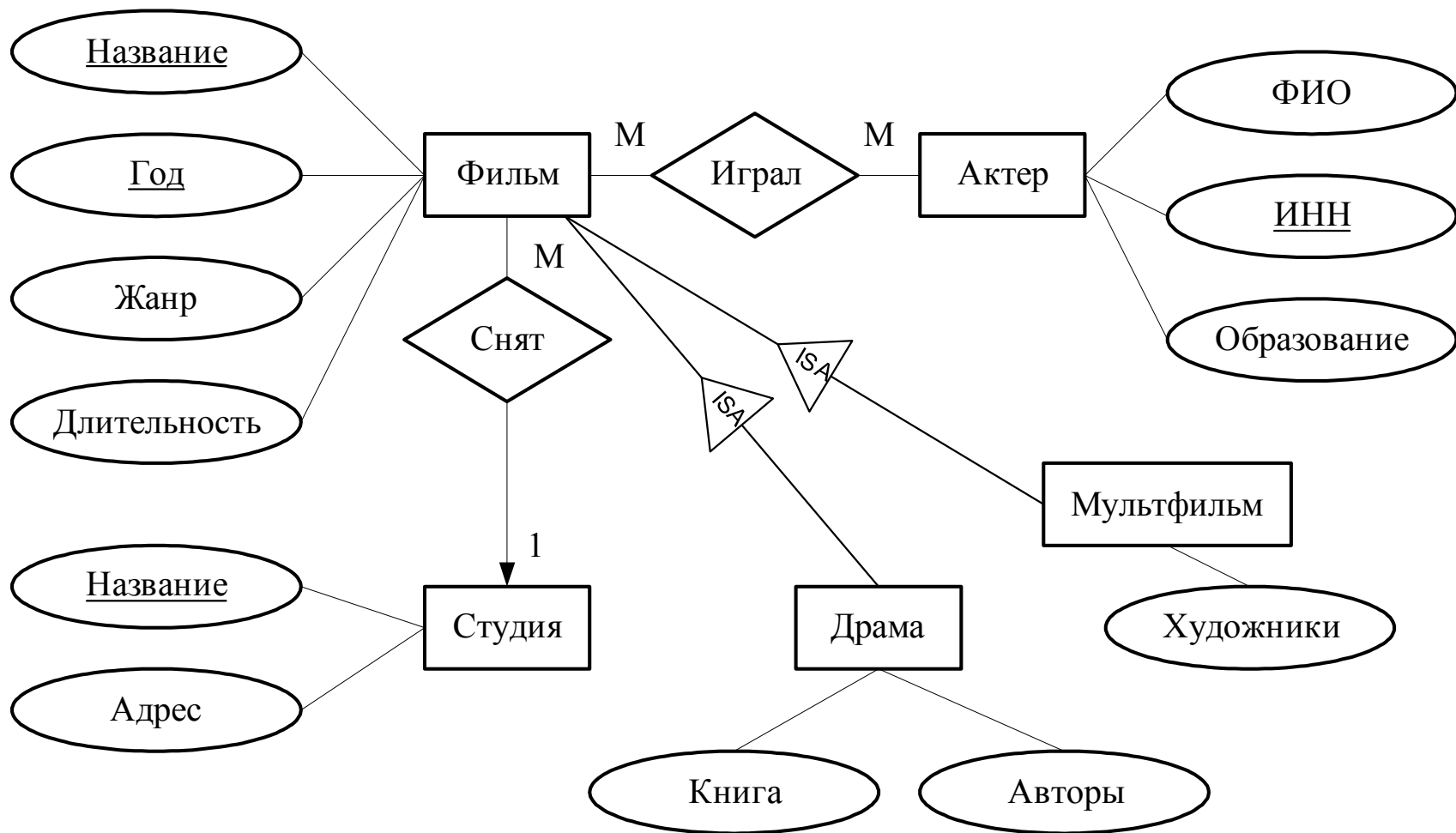
- Дисциплина (Название, Лектор)
- Группа (Шифр, Кафедра)
- Аудитория (Номер, Корпус)
- Занятие (Дисц (*Дисциплина),
Группа(*Группа), Ауд (*Аудитория))



- Кафедра (Название, Направление, вуз(*Вуз))
- Вуз (Название, Адрес)

Преобразование иерархии сущностей (в ОРМ)

- Связь ISA задают через наследование типов.
- Каждой производной сущности присваивают свое отношение, включающее все атрибуты базовой сущности и ее собственные, и указывают название базовой сущности.



Актёры (ИНН, Ф.И.О (Фамилия, Имя, Отчество),
Образование)

Фильмы (Название, год, Длительность, Жанр,
Студия (*Студии))

Студии (Название, Адрес (Город, Улица))

Актёр–Фильм (Фильм (*Фильмы), Актер (*Актеры))

Мультфильмы: Фильмы (Название, Год,
Длительность, Жанр, Студия (*Студия),
Художники)

Драмы: Фильмы (Название, Год, Длительность,
Жанр, Студия (*Студия), Книга, Авторы)

Пример наследования

```
CREATE TYPE AnimationT under FilmT AS  
(  
    drawers ARRAY[varchar(70)]  
);
```

```
CREATE TYPE SerialT under FilmT AS  
(  
    book varchar(100),  
    avtors ARRAY[ varchar(20) ]  
);
```


Пример наследования таблиц

```
CREATE TABLE TF OF FilmT
```

```
(
```

```
  REF IS Idf system generated
```

```
  PRIMARY KEY (title, year),
```

```
);
```

```
CREATE TABLE Animation of AnimationT under  
  TF ( );
```

```
CREATE TABLE Serial of SerialT under TF ( );
```

Примеры запросов

Фильмы, мф, сериалы:

- `SELECT * FROM TF`

Только мф:

- `SELECT * FROM Animation`

Фильмы (без мф и сериалов):

- `SELECT * FROM ONLY TF`

Правила сравнения пользовательских типов данных

- Поэлементное сравнение на равенство для объектов типа StudT:

**CREATE ORDERING FOR StudT
EQUALS ONLY BY STATE**

- Полное правило сравнения через функцию для пользовательского типа SerialT:

**CREATE OREDERING FOR SerialT
ORDER FULL BY RELATIVE WITH Fun**

Результат функции сравнения $\text{FUN}(x_1, x_2)$

- Для функции равенства
 - 0, если $x_1 = x_2$
 - Не 0, если $x_1 \neq x_2$
- Для функции полного сравнения
 - 0, если $x_1 = x_2$
 - Значение < 0 , если $x_1 < x_2$
 - Значение > 0 , если $x_1 > x_2$

Функция сравнения типов

```
CREATE FUNCTION Fun (IN S1 SerialT, IN S2 SerialT)  
  RETURNS integer
```

```
IF S1.length()<S2. length() THEN RETURN (-1)  
  ELSEIF S1.length()>S2.length() THEN RETURN (1)  
  ELSEIF S1.year()<S2.year() THEN RETURN (-1)  
  ELSEIF S1.year()>S2.year() THEN RETURN (1)  
  ELSEIF S1.title()<S2.title() THEN RETURN (-1)  
  ELSEIF S1.title()>S2.title() THEN RETURN (1)  
    ELSE RETURN(0)
```

```
ENDIF;
```

Работа с UDT

- Переход по ссылке вместо соединения таблиц
- Разыменование ссылки
- Использование методов генераторов (конструктор), модификаторов (set) и обозревателей (get)

Переход по ссылке (1-M)

```
SELECT year, length,  
        stud->sname,  
        stud->addr.city,  
        stud->addr.fullad()  
FROM TF  
WHERE title = 'The Matrix'
```

Переход по ссылке (M-M)

```
SELECT film->title,  
        film->year,  
        film->stud->name  
FROM FA  
WHERE act->fio='Иванов'
```


Разыменование ссылки

```
SELECT deref( act )  
FROM FA  
WHERE film->title='The Matrix'
```

Использование метода-обозревателя

- Нельзя обращаться к атрибутам по конкретным именам,
- вместо этого используют метод-обозреватель (он вызывается при обращении к атрибуту)

```
SELECT title()  
FROM TF  
WHERE type() < > 'комедия'
```

Использование методов генераторов и модификаторов

DECLARE newA Addr; — объявим переменные
DECLARE newS Stud;

SET newA = Addr(); — вызовем генератор
newA.city('Москва'); — вызовем модификатор
newA.street('Ленина, 2');
SET newS = Studia(); — вызовем генератор
newS.sname('Мосфильм'); — модификаторы
news.addr(newA);

INSERT INTO TSt VALUES(newS);
 — добавим объект в таблицу

Спасибо за внимание!

Виноградова Мария Валерьевна

Vinogradova.m.iu5@bmstu.ru

МГТУ им. Н.Э. Баумана
кафедра Систем обработки информации и
управления (ИУ5)